Lab 3: Inverting an image

Phillip Pinter

March 7, 2024

1 Introduction

The goal of this lab session is to first make a kernel that inverts an image ($x = 255 - x$, where $x \in (0, 255)$, for each color component) and then augment that kernel so the red component of the image is changed according to the following formula:

$$x = (x\%25) \cdot 10, \quad x \in (100, 200)$$

In the first part, the influence of the number of threads (full/partial warps) on the computation time is tested. In the second part the impact of thread divergence on the computation time is tested.

2 Results

2.1 Part1: Warps

For this part, the computation time is compared as follows: change the thread to element ratio to assess how the total amount of threads impact the time. Each case is done 999 times (an uneven number so after inverting each time, the result is the inverse of the original image) and the averages of those tests are displayed. Following ratios result in a total amount of threads that's a multiple of 32 (full warps) which were found experimentally by testing every number between 1 and 500: 1, 2, 3, 6, 11, 22, 33 and 66. The other ratios are just conventional ratios where the previous ratio is doubled, starting from 1.

```
Threads per element ratio: 1:1          Average Time: 0.0845611 ms
Threads per element ratio: 1:2          Average Time: 0.0626323 ms
Threads per element ratio: 1:3          Average Time: 0.0383143 ms
Threads per element ratio: 1:4          Average Time: 0.0380061 ms
Threads per element ratio: 1:6          Average Time: 0.0432175 ms
Threads per element ratio: 1:8          Average Time: 0.0351032 ms
Threads per element ratio: 1:11         Average Time: 0.0373538 ms
Threads per element ratio: 1:16         Average Time: 0.0388775 ms
Threads per element ratio: 1:22         Average Time: 0.0349861 ms
Threads per element ratio: 1:32         Average Time: 0.0337481 ms
Threads per element ratio: 1:33         Average Time: 0.0326659 ms
Threads per element ratio: 1:64         Average Time: 0.0355408 ms
Threads per element ratio: 1:66         Average Time: 0.0333657 ms
Threads per element ratio: 1:128        Average Time: 0.0569859 ms
```

Here we can see that using one thread for each element isn't necessarily faster and using less threads could actually be beneficial for performance (in this test, using 1 thread for each element is even the slowest). This is due to the better utilization of each thread and reduced overhead from managing a large number of threads. This trend goes on until the ratio of 1:33 which is the most optimal ratio in this test. After this, the performance decreases again, possibly because each element is processing a large amount of elements now and the GPU is underutilized for parallelization.
In previous test where the full warp ratios weren't , the ratio 1:32 was the fastest (this is still visible in the results above), this seemed to be the sweet spot between parallelization and per-thread workload but since the total amount of threads isn't a multiple of 32, it could be further optimized. The ratio 1:33 does result in a full warp and results in the fastest computation time.

2.2 Part2: Thread divergence

For this part, the green and blue components are still inverted but the red component is calculated by following formula:

$$x = (x\%25) \cdot 10, \quad x \in (100, 200)$$

This is done in 2 different ways:

1. Following input format is processed (original format): RGBRGB...RGB
2. Following input format is processed (reformatted): RR...RGGG...GBB...B

Each case is again done 999 times to find the average time.

```
Average time for original kernel:    0.135916 ms
Average time for reformatted kernel: 0.067535 ms
```

These results show the advantage of minimizing thread divergence, the reformatted input is more than twice as fast. Thread divergence occurs when different threads within the same warp take different execution paths which can impact performance negatively. In the original format of "RGBRGB...RGB", there is a big chance a warp has threads which need to take a different execution path for the red components which impact the performance negatively.
By reformatting the input to "RR...RGGG...GBB...B", each warp of 32 threads has a much bigger chance of having all 32 threads processing elements of the same color so there is only one execution path in that warp.

3 Result images

The resulting images of the first and second part are displayed below.



Figure 1: Part1 - Warps



Figure 2: Part2 - Thread divergence